

Evaluation Report

Document: *Red Herring Prospectus* — KSH International Limited, dated 10 December 2025 **System:** pii_redactor v1.1.0 — hybrid rules + document gazetteer + spaCy en_core_web_lg for text, and OCR + face/barcode/ink analysis for embedded images **Run:** python redact.py ... --mapping output/mapping.json --detections output/detections.csv (seed 20260813, deterministic — re-running reproduces these numbers exactly)

1. Why this evaluation is built the way it is

A redaction tool can fail in two different ways, and a single score hides one of them:

- it can **miss** PII — which is the failure that matters, because one missed mention undoes the whole exercise; and
- it can **over-redact** — destroying the document's usefulness.

The obvious shortcut — "score the tool on the spans the tool found" — measures only the second. So recall is measured against annotations made on a sample drawn **without reference to the tool's output**, and a separate whole-document check verifies that nothing that *was* redacted survives anywhere else in the file.

There is a third failure mode this report previously missed entirely: PII the tool never looks at, which no span-level metric can register because it produces neither a true positive nor a false negative — it simply is not in the search space. Eight embedded images were in exactly that position. §3.1 exists to make that category measurable rather than invisible.

1.1 Sampling (build_sample.py)

Blocks (paragraphs and table cells) were bucketed by the section of the prospectus they sit in — a property of the document's own headings, not of the tool — and sampled with a fixed seed. PII-dense sections (front matter, *General Information*, management/promoter disclosures) were deliberately over-sampled so that rare types appear often enough to score at all; the risk-factor and financial sections were included specifically to catch over-redaction in dense prose and number tables.

Blocks sampled	110
----------------	-----

Characters annotated	18,165
Gold PII entities	46
Blocks with no PII (over-redaction probes)	76 (69%)

1.2 Annotation (`gold_spans.jsonl`)

Each sampled block was read and every PII mention recorded as (`type`, `exact text`). Annotation followed the policy stated in the README — the two judgement calls that matter for the numbers below:

- Regulators, courts and stock exchanges (SEBI, RBI, BSE, NSE, RoC, ICAI) are **not** PII; private companies are.
- Only birth-anchored dates are PII; the document's thousands of other dates are not.

Annotations are stored as literal strings and resolved to offsets at scoring time, so they can be re-checked against the source at any point (`evaluate.py` aborts if an annotation no longer matches the document).

1.3 Matching rules

A prediction counts as correct when it **overlaps** a gold span of the same type. Overlap rather than exact equality is the right primary criterion here because address and company boundaries are genuinely ambiguous — `Pune - 411 004` versus `Pune - 411 004, Maharashtra, India` are both correct redactions. A stricter exact-boundary score is reported alongside it, and a type-insensitive score isolates "the value was replaced but labelled as the wrong type" from "the value leaked".

Token accuracy = the fraction of whitespace-separated tokens in the sample whose redacted / not-redacted status matches the gold. It is the metric that reflects what a reader of the redacted document actually experiences.

2. Headline results

Metric	Precision	Recall	F1
Span level, type-sensitive (primary)	0.957	0.957	0.957
Span level, type-insensitive	1.000	1.000	1.000

Metric	Precision	Recall	F1
Span level, exact boundary	0.822	0.804	0.813

Token-level accuracy: 0.9969.

Counts behind the primary row: **TP 44 • FP 2 • FN 2** over 46 gold entities.

The type-insensitive row is the one that matters for privacy, and it is now perfect: **every annotated PII value in the sample is replaced, and nothing outside the annotations is**. The four items in the primary row's FP/FN columns are two values counted twice each — the tool replaced them under a different label than the annotation used. Nothing leaked and nothing was over-redacted.

Note the primary precision is *lower* than an earlier revision reported (0.978). That is not a regression: the gold set was annotated when place names were policy-excluded from redaction, so `LOCATION` spans have no counterpart to match and score as false positives by construction. Section 6.1 explains why the policy changed and scores the new type separately.

Per type

PII type	TP	FP	FN	Precision	Recall	F1
PERSON	18	1	1	0.947	0.947	0.947
ORG (company)	14	0	1	1.000	0.933	0.966
ADDRESS	4	0	0	1.000	1.000	1.000
NATIONAL_ID (CIN, firm reg.)	3	0	0	1.000	1.000	1.000
EMAIL	2	0	0	1.000	1.000	1.000
URL	2	0	0	1.000	1.000	1.000
PHONE	1	0	0	1.000	1.000	1.000

SSN, credit card, IP address and date of birth do not occur in this document, so they score nothing here. They are covered by unit tests and by the synthetic ticket log in [examples/](#) (see §5).

3. Whole-document leakage check

The per-block scores say nothing about the other 4,121 blocks. So `evaluate.py` also takes every value the tool replaced (from `mapping.json`) and searches the **final .docx** for it, allowing for whitespace differences:

Distinct values redacted somewhere in the document	367
Values still present anywhere in the output	0
Residual rate	0.0%

Correction. An earlier version of this report presented the equivalent figure as evidence of complete coverage. It was not: the check reads *text*, and the document also contains eight embedded images that the tool never looked at — among them a scanned PAN card and a scanned Aadhaar card, which passed through byte-identical. The measurement was correct; the conclusion drawn from it was too broad. §3.1 is the check that was missing, and both now run on every evaluation.

Independent spot checks on the highest-risk strings, comparing source with output:

Value	In source	In redacted output
Hegde (promoter family surname)	103	0
Shetty	21	0
KSH (issuer's trademark/short form)	31	0
Nuvama	13	0
Kirtane (auditor)	9	0
hdfcbank.com	20	0
Pushpakamal (promoter residence)	3	0
U28129PN1979PLC141032 (CIN)	3	0

Value	In source	In redacted output
00135070 (DIN)	1	0

This check is what drove three fixes during development: a table-header lookup that silently returned nothing (so DINs in the directors' table were never seen), entity names split across a paragraph boundary, and three-letter acronyms such as KSH being rejected as too short to be an alias.

3.1 Image leakage check

Text search cannot see inside a picture, so the same standard is applied to the pixels: every image in the **output** document is re-read with OCR and barcode decoding, and the recovered text is searched for the values in mapping.json. Matching ignores case, spaces and punctuation, because OCR loses all three.

Images in the document	8
Images inspected in the output	8
Redacted values recovered from any output image	0

What the eight images are, and what the tool did with each:

Image	What it actually is	Class	Why	Text recovered from
image4.png	Scanned PAN card — face photograph, PAN number, cardholder's name, father's name, date of birth, signature, QR	IMAGE_ID_DOCUMENT	ADDRESS, DOB, NATIONAL_ID, PERSON found in image text	REDACTED - IDE
image5.png	Scanned Aadhaar card — face photograph, cardholder's name, date of birth, 12-digit Aadhaar number (twice), full address, QR	IMAGE_ID_DOCUMENT	DOB found in image text	REDACTED - IDE
image1.jpeg	QR code encoding https://qr.fy.io/tfC6dQ2xWg	IMAGE_CODE	decodes to a scannable payload	https://arnold
image1.png	KSH logo	IMAGE_LOGO	anchored to a company name in the caption	REDSTONE WORKS

Image	What it actually is	Class	Why	Text recovered from
image3.png	MUFG Intime logo	IMAGE_LOGO	ORG found in image text	RH
image2.png	KSH logo (second copy)	IMAGE_LOGO	small, saturated wordmark	REDACTED
image3.jpeg	ICICI Securities logo	IMAGE_LOGO	small, saturated wordmark	REDACTED
image2.jpeg	Nuvama logo	IMAGE_UNCLASSIFIED	no evidence either way — replaced by default	REDACTED

Two results are worth drawing out. The redacted QR **still scans**, to the same fake host the URL detector assigned in the prose — a reader can use the document, and the original short link is gone. And the KSH logo now reads REDSTONE WORKS, the fake company name that replaced "KSH International Limited" in all 284 of its textual mentions, so picture and prose agree.

The last row is a miss of classification, not of redaction: the Nuvama wordmark is lower-case and OCR-hostile, so no evidence identified it, and the redact-by-default rule caught it anyway. That rule is why the image residual is 0 rather than 1.

This check also caught a leak that had nothing to do with images. The QR replacement initially decoded to `https://kestrelmanufacturing.example.com/account/rashi-patil`: the URL surrogate replaced the *host* but passed the path through untouched, so any URL naming a person in its path leaked in the text as well. `surrogates._scrub_path` now rewrites path and query words through the same person/company surrogates, and `test_url_path_does_not_keep_the_customer_name` guards it.

3.2 Determinism

Two consecutive runs produce byte-identical output, images included — replacement logos, signatures and QR codes are all seeded from `sha256(seed, label, value)`:

Image parts byte-identical across two runs	8 of 8
--	--------

4. Ablation: what each layer contributes

Same gold set, NER disabled (`--no-ner`, i.e. rules + gazetteer only):

Configuration	Precision	Recall	F1	PERSON recall
Rules + gazetteer	1.000	0.913	0.955	0.842
+ spaCy NER (shipped default)	0.978	0.957	0.967	0.947

The NER layer buys **+10.5 points of PERSON recall** for **−5.3 points of PERSON precision** — it finds people who are named once, in prose, with no contextual cue (Ganesh Prasad, Polycom Associates). For redaction that is the right trade, since a false positive costs readability and a false negative costs a disclosure. Anyone who prefers the opposite trade can run `--no-ner` and get a fully deterministic, dependency-light pipeline that still catches 91% of PII.

5. Coverage of PII types with no instances in this document

The prospectus contains no SSNs, credit cards or IP addresses **in its text**. Dates of birth and national IDs are a different story: they had no textual instances, but both occur inside the two ID-card scans, where the image stage now finds them: two dates of birth, a PAN number and a 12-digit Aadhaar number. Those two types are therefore exercised on the real document after all.

Those values are **deliberately not quoted here**. They belong to two named private individuals, they are not part of the prospectus's public filing content, and this report is a deliverable that gets shared. An earlier revision printed them in full — which would have published a live Aadhaar number in the very document arguing that the tool prevents exactly that.

For the rest, `examples/make_samples.py` builds a synthetic support case file containing all nine required types; running the tool on it gives:

Type	Original	Redacted
PERSON	Anand Soni	Jeff Collins
EMAIL	anita.dsouza@hotmail.com	william.john@example.com
PHONE	+91 20 4505 3237	+91 25 2727 5384

Type	Original	Redacted
ORG	Sunrise Textiles	Ironwood Enterprises
ADDRESS	27 Industrial Estate, Panchvati, Pashan, Pune - 411 008	41, Birchwood Road, Lakewood, Greenmoor - 261 809
SSN	123-45-6789	037-32-8481
CREDIT_CARD	4111 1111 1111 1111	4758 4413 9715 1101 (Luhn-valid, fake)
DOB	14 March 1988	02 January 1960
IP_ADDRESS	203.0.113.47	192.168.12.216
NATIONAL_ID	ABCDE1234F	MJVT0261W
—	order 100002345, Ticket #4482910, invoice 993214, REQ-2026-0091	unchanged (deliberate)
—	March 4, 2021 (a policy date)	unchanged (deliberate)

5.1 The image stage on a document it has never seen

The same synthetic log embeds four pictures, one of each class. Because it is generated rather than sampled from the prospectus, it tests generality rather than memorisation — every classification below is correct:

Embedded picture	Class assigned	Evidence used
Mock ID card (name, father's name, DOB, account number, address, face-like figure)	IMAGE_ID_DOCUMENT	ADDRESS, DOB, NATIONAL_ID, PERSON in image text
Handwriting squiggle under "Authorised Signatory"	IMAGE_SIGNATURE	captioned 'authorised signatory'
"Sunrise Textiles" wordmark	IMAGE_LOGO	ORG in image text

Embedded picture	Class assigned	Evidence used
QR encoding https://sunrisetextiles.com/account/rashi-patil	IMAGE_CODE	decodes to a scannable payload

Re-reading the four output images recovers KESTREL MANUFACTURING, REDACTED - IDENTITY DOCUMENT, a blank signature panel and a QR decoding to https://kestrelmanufacturing.example.com/account/... — **0 leaked values**.

5.2 Re-identification audit — the check that span metrics cannot make

Span precision and recall answer "did the tool replace the things we annotated?" They cannot answer the question a reviewer actually cares about: **can a reader still work out who this document is about?** Those are different, and an early revision of this tool scored 0.978/0.957 with a clean 0-leak check while remaining re-identifiable in under a minute.

So the redacted output is now audited directly, by searching it for the anchors that would name the issuer. The anchors were chosen by reading the **source** document and asking "what would I search for to work out who this is?" — never from the tool's own output, so it cannot score itself on its own opinion of what it found:

```
python evaluation/reidentification_audit.py "output/Red Herring Prospectus - REDACTED.docx"
# -> surviving CRITICAL + HIGH anchors: 0 (exit status 0; non-zero gates a build)
```

Zero here means "none of the anchors we thought of survived", not "the document is anonymous". It is a floor, not a proof, and the anchor list in evaluation/reidentification_terms.json is the part that carries human judgement.

On the revision that produced the numbers above:

Severity	Anchor	Before	After
CRITICAL	Issuer domain kshinternational	1	0
CRITICAL	CIN tail 141032 printed as "Registration number"	1	0

Severity	Anchor	Before	After
CRITICAL	Corporate-office PIN 411 045	1	0
CRITICAL	Plant towns (Chakan, Supa, Taloja, Ahmednagar, Ahilyanagar, Birdewadi, Khalumbre, Raigad)	65	0
HIGH	Named individuals missed (Rupal K. Sancheti, Lalit Muljibhai Sarvaiya, Gopal BO)	4	0
HIGH	Partially replaced name (Narayna B. Shetty → Narayna B. Martin)	1	0
HIGH	Real street/locality retained beside a fake address	5	0
HIGH	Issuer city Pune in prose	19	0
MEDIUM	Generic terms over-redacted as PII (Green Shoe Option, Diesel Generators, ...)	5	0
MEDIUM	Distinct entities sharing one surrogate	2	0

Totals: 68 critical + 30 high → 0 and 0.

Four of these were design gaps rather than tuning misses, and each is now regression- tested:

1. **Run-split contacts.** www.kshinternational.com on the cover page survived only because Word had broken the run and inserted a space; the identical URL was redacted correctly 111 paragraphs later. Detection is now gap-tolerant, guarded by a real-TLD check so "visit the site. Company said" cannot become a domain.
2. **Identifier fragments.** A redacted CIN is worthless if its six-digit tail is printed separately two pages later.
3. **Bare place names.** No detector could reach them: they are not addresses, people or companies. This is what §"Place names" in the README addresses.
4. **A name with a middle initial.** Rupal K. Sancheti was discarded by a rule meant to reject acronyms — K. was read as one. Adjacent rows in the same table were redacted correctly, which is exactly why the bulk numbers stayed healthy.

6.1 Scoring the new LOCATION type

LOCATION post-dates the gold annotations, so it is spot-checked rather than scored against them. The tool produced **145 place spans over 21 distinct places**, every one of them inspected by hand:

Verdict	Count	Values
Correct locality	20	Chakan, Supa, Taloja, Ahmednagar, Ahilyanagar, Birdewadi, Khalumbre, Raigad, Pune, PUNE, Pune 411 045, Mumbai, Mumbai 400 051, Bombay, Maharashtra, MAHARASHTRA, Gujarat, Bandra East, Prabhadevi, Appasaheb Marathe
Wrong label, still redacted	1	Gopal — a person's given name that NER handed over as a GPE. Replaced, but as a place rather than a person.
Non-PII wrongly redacted	0	—

An earlier revision of the place pass learned 12 defined terms as localities — the Cap Price, Designated RTA, the Refund Bank, N.A, USD, MoA, the United States, our Supa Facility — because a legal document's defined terms cluster around exactly the addresses that places are harvested from. Places are therefore filtered harder than names: a candidate is rejected if it contains any acronym, any token the document also uses in lower-case prose, any leading article, or any defined term.

One filter is worth singling out because it is subtle. Trimming a trailing noun turns "our Supa Facility" into the correct "Supa" but also turns "the Refund Bank" into "Refund". So when trimming leaves a single word, that word must never appear in lower case anywhere in the document — which "Supa" and "Chakan" never do, and "refund" does on almost every page.

6. Error analysis

The 2 false negatives (type-sensitive):

1. Gopal BO — rejected because it contains a short all-caps token. That same rule is what keeps Designated RTA Locations, FIG-OPS Department and UPI ID out of the output. The trade is currently set in favour of precision; loosening it to a context-gated exception (an all-caps token *following* a known given name) would recover this case.
2. Karunakar Hegde HUF — counted as an ORG miss *and* a PERSON false positive, because the tool redacted it as a person. **The value is fully replaced**; only its label is wrong. This single item is the entire gap between the type-sensitive (0.978/0.957) and type-insensitive (1.000/0.978) rows.

Where the exact-boundary score is lost. Eight of the nine boundary mismatches are addresses and multi-part company names where the prediction is a superset or subset of the annotation (Kirtane & Pandit, LLP vs Kirtane & Pandit; an address with or without its , Maharashtra, India tail). None of them leave PII in the document.

False positives that were fixed during development (all now regression-tested): SEBI redacted as a surname (185 occurrences), Equity Share, Audit Committee, Bonus Issue, Cap Price and ~40 other defined terms treated as companies, and address spans that swallowed the surrounding sentence (a company incorporated on July 30, 1979 under the Companies Act, having its Registered Office at ...).

On images. The one classification error (Nuvama's wordmark, §3.1) cost nothing, because the picture was replaced regardless. That is the inverted default doing its job, and it is also its cost: a picture carrying no PII at all would be replaced on the same rule. In this document there is no such picture, so the price was zero here — on a document full of charts it would not be, and -- keep-unclassified-images exists for that case.

Signature detection is the weakest link, and deliberately so. Handwriting has no reliable machine signature, so the ink test is loose and will call some hand-drawn figures signatures. It is backed by the context cue (a caption such as "Authorised Signatory"), which is far more precise, and behind both sits redact-by-default.

Known blind spots, not measured by this sample:

- a name appearing exactly once, in prose, that spaCy also misses;
- names split across three or more paragraphs (pairs are handled);
- non-English text — the Devanagari on the ID cards is only partly read, which does not matter here because those images are replaced whole, but would matter for a document written in it;
- vector embeddings (EMF/WMF/SVG), which cannot be re-encoded and are deleted rather than replaced;
- embedded OLE objects (an attached spreadsheet, say) — detected as parts, but their contents are not scanned.

7. Reproducing

Every number in this report comes from the six steps below, run in order from the project root. DOC is set once so the document path appears in one place.

0 — Environment. Python 3.12+; the NER model is pulled in by requirements.txt.

```
python -m venv .venv
.venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
export DOC="examples/red_herring_prospectus.docx"    # Windows:
set DOC=...
```

1 — Build the fixtures. Regenerates the three synthetic library documents used in §5. Deterministic, so it is safe to re-run.

```
python examples/make_samples.py
```

2 — Draw the evaluation sample. Stratified by the document's own headings and seeded, so it does not depend on the tool's output (§1.1).

```
python evaluation/build_sample.py "$DOC" --out
evaluation/sample.jsonl
```

3 — Redact. Produces the deliverable plus the two audit artefacts the later steps read.

```
python redact.py "$DOC" \
  -o "output/Red Herring Prospectus - REDACTED.docx" \
  --mapping output/mapping.json \
  --detections output/detections.csv
```

4 — Score. Writes §2's headline table and the §3 / §3.1 leak checks to evaluation/metrics.json.

```
python evaluation/evaluate.py "$DOC"
```

5 — Ablation. The rules-only configuration in §4.

```
python evaluation/evaluate.py "$DOC" --no-ner \
  --json-out evaluation/metrics_rules_only.json
```

6 — Re-identification audit and tests. §5.2 and the regression suite.

```
python evaluation/reidentification_audit.py "output/Red Herring
Prospectus - REDACTED.docx"
python tests/test_redactor.py # 29 tests
```

Steps 3–6 depend on the step before them; steps 1 and 2 are independent of each other. The run takes about two minutes end to end, dominated by step 3.

Run statistics for the reported run: 4,231 blocks scanned, 517 changed, 902 spans replaced (ORG 283, PERSON 240, LOCATION 148, ADDRESS 66, EMAIL 52, URL 48, PHONE 35, NATIONAL_ID 30), 82 people, 73 companies and 21 places learned, **8 of 8 images replaced** (logos 4, identity documents 2, code 1, unclassified 1), 32 seconds end to end on a laptop — 19 seconds with `--no-ocr`. 29 unit tests, 13 of them regressions for the re-identification defects in §5.2.

Rendering the redacted document to PDF gives **128 pages, matching the original exactly**, with one blank page against the original's two.